DESIGN BREAKDOWN:

Screens:

- Monster editor main menu
    - Options for (1. Browse monsters by dungeon level) and (2. Search monster by name)
    - Option q - quit

- Browse monster by dungeon level screen (navigated to by pressing 1)
    - Options for (1. Levels 0-10), (2. Levels 11-20),..., (0. Levels 91-100)
    - Option q - quit

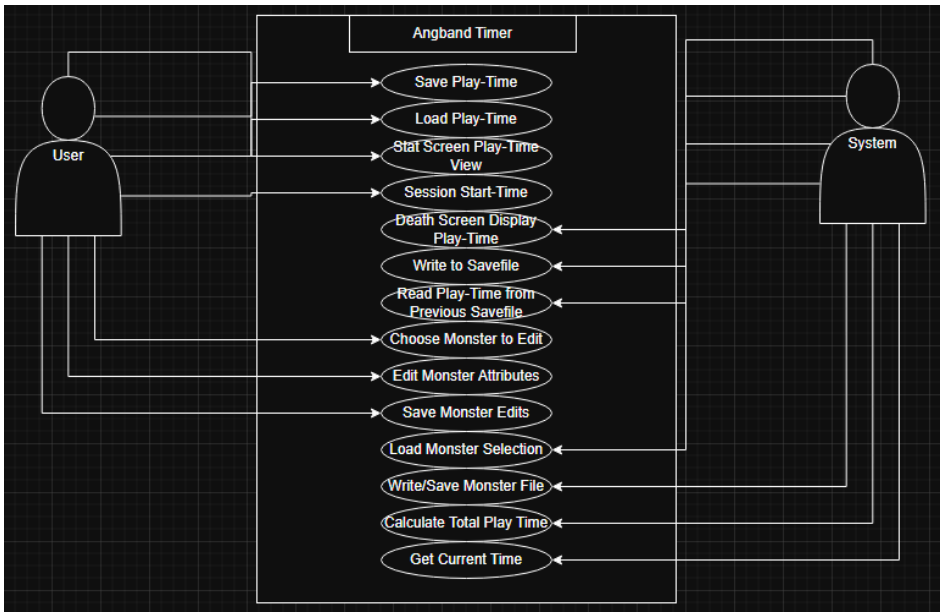
    - Level specific search screen (navigated to by pressing a certain number)
        - List of monsters appearing first in specified levels (i.e. Blubbering Idiot (Level 0)
        - Option q - quit

- Search monster by name screen (navigated to by pressing 2)
    - Monsters in alphabetical order (show 10 per page)
    - Options p - previous page, n - next page, q - quit

- Monster edit screen (navigated to by selecting certain monster from name screen or level specific search)
    - (1. Name), (2. Speed),..., (9. Additional attribute 1 – armor class), (0. Additional attribute 2 - dodge)
    - Options q - quit, s - save

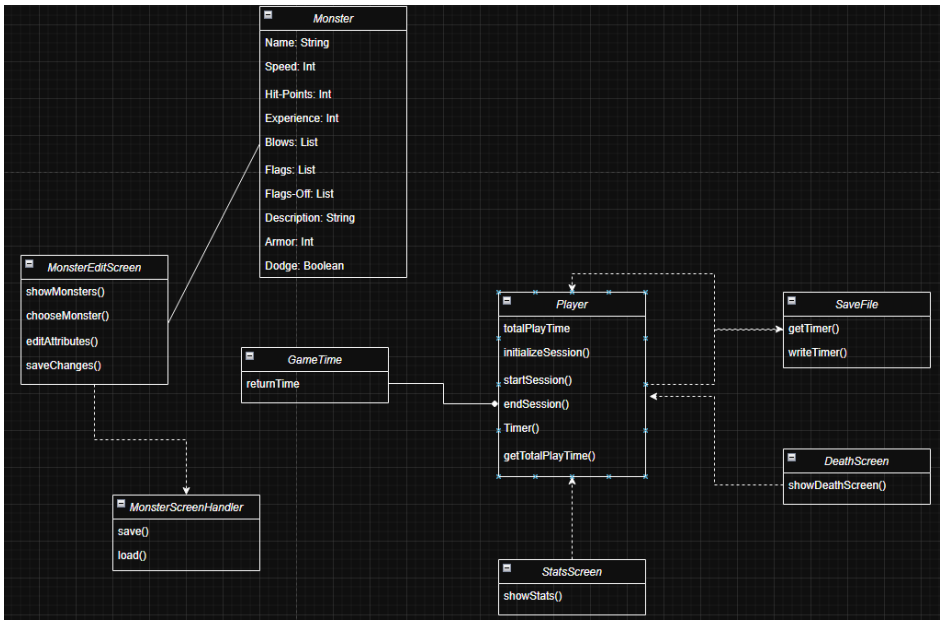
- Game character stats with play-time display
- Death screen with play-time display

Use Cases:



Save Play-time is triggered when PLAYER saves/exits, and Load Play-time is used when PLAYER resumes a saved game. Start screen play-time view displays play time while PLAYER is on the main/start screen, and Session start-time records the exact time when PLAYER begins play. Death screen display time SYSTEM shows play time when the character dies. SYSTEM also stores updated play-time into data via Write to Savefile, and SYSTEM retrieves stored play time from an existing save file during loading via Read Play-Time from existing savefile. For monster editing, PLAYER utilizes Choose Monster To Edit for selecting a monster entry to modify, and Save Monster Edits to save changes to that registry. PLAYER can use Edit Monster Attributes to modify stats such as HP, Damage, Speed, and other attributes. SYSTEM will Load Monster Selection which loads a previously selected monster editing, and saves monster data into a file after edits via Write/Save Monster File. Additionally, SYSTEM will retrieve the current system clock time per tick and calculate session duration.

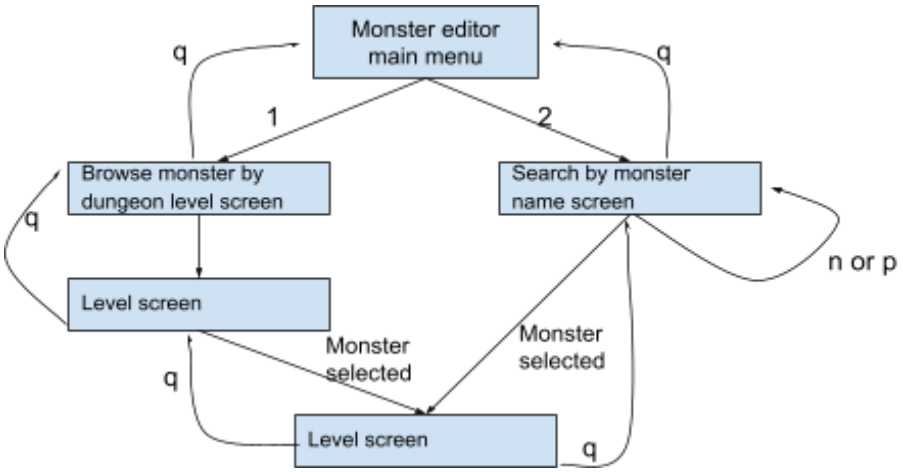
Class Diagram:



The player section will be using the player.h/player.c files which will store the timer fields such as the initializeSession(), startSession(), endSession(), Timer(), and getTotalPlayTime(), essentially managing the timer, and computing the play time using returnTime. The GameTime will represent the clock and will provide the returnTime. The SaveFile is handling the save/load behavior and using the files savefile.c, save.c and load.c, then written during the save. When the game loads, the getTime() adds to the totalPlayTime, then writeTime() updates the value after the session has ended. The timer is displayed on the StatsScreen which will use the ui-player.c file and use showStats() to get the current value using getTotalPlayTime(), then display that value on the character stats screen. The DeathScreen uses the file ui-death.c which will use the showDeathScreen() to finalize the game session and display the total play time when the user's character dies. The monster editor contains all the attributes. The MonsterEditScreen uses the methods showMonsters(), chooseMonster(), editAttributes(), and saveChanges(), that collects the user input and updates the monster objects. MonsterScreenHandler will prove the load() and save() methods that parse monster data into the monster objects and handle the updated monster data back to the file. The user will modify the monster fields through editAttributes(), and saveChanges will start the save().

Flow Diagram:

- Navigation between monster editor screens



Major Task Items:

- Parse files for information
- Manage data for ea
- Classification logic for grouping monsters by dungeon level
- Alphabetizing logic for search monsters by name
- Data updating for edited monster characteristics
- Validating monster edits

<u>TEST PLAN DEVELOPMENT</u>

This part of the document will cover the test plan for our *Angband* monster editor. Before going over major and edge test cases, there is some set up that will need to be tested first. These include:

1. Ensuring the program can be read in the monster.txt file as well as the other related game files without errors or crashing.
2. Ensure all monsters can be searched within the program, and that they appear under their appropriate dungeon level.
3. Ensure all features of the editor work as intended and do not crash or create errors. Also that when a user makes a change in the editor, that change is reflected in the newly written file.

4. Ensure when the user is finished they are able to write a new file, which the changes they made included, and that this text file works with *Angband* with the monsters being updated with those changes.

Once these necessary set up steps are completed, our team will move on to testing edge cases and sample test cases, to ensure the program works as intended. Our main methods of testing will be unit and integration testing. These two are the most important as unit testing ensures each of the functions and individual components of the program work as intended such as being able to edit monster stats, edit multiple monsters, search monsters, and write out their changes to a new file. These must work in isolation before moving on to integration testing to ensure that our editor program's components are all functioning with each other, as well as with *Angband*. The team must do both these methods of testing to ensure both individual and group parts of the program work as intended. Listed below are several sample test cases we will be checking on our editor.

(This table will be updated to include actual results when testing officially begins.)

Test Case ID	Description	Steps	Expected Result
1	Invalid input for changing a stat of a monster, or when searching a monster.	Within the editor, using the search bar/when inputting a value on the monster edit screen.	The program will output a invalid input message, removing the input and allowing the user to reattempt.
2	Doing multiple edits to a monster's stats, or multiple edits to multiple monsters.	Within the editor when changing the monster's stats, and then leaving that page and going to another monster and editing that one's stats.	The editor will store this changed information, ensuring it is reflected in the new text file when the user is finished.
3	Changing a single stat on a single monster.	Choosing a monster either from the dungeon list or search feature, taking the user to the edit page and allowing them to change a stat.	The changed stat will be stored and reflected in the written text document, with the change reflected in *Angband*.
4	Monsters show up when searched on	Navigating to the search page, and tying in a	The monster appears in the search list after its

	the search screen.	monster name correctly.	name has been inputted.

TIME ANALYSIS AND ROLE ASSIGNMENTS :

This document is an assessment of the time it takes to alter the Angband game to fit the specifications of the designers. The task has been broken down into different parts with assignments due date etc.

Angbang Time estimate						
Project: Angband Timer and monster search			Date: 02/12/2026			
Activity	Assignments: (can be adjusted as needed)	Optimistics	Most Likely	Pessimistic - Potential hang-ups	Time Reserve:	Calendar due date:
Parsing Files For information	Elyse	.5 Hours	1 hour	1.5 Hours- Large amount of files to root through	1 hour	February 15th
Data Allocation in C and its management- plan	Emily Sergio	2 Hours	2.5 Hours	3 hours- Memory allocation across multiple files	2 hours	February 16th
Classification logic and grouping monsters by dungeon level. Navigation and group by levels- Options for (1. Levels 0-10), (2. Levels 11-20),...,	Chris Connor	3	3.5	4 hours- Time it takes to reclassesify each monster already existing in monster.txt file, and add a level associated to it	3.5	February 18th

(0. Levels 91-100) ETC..						
Monster search by name	Elyse	5	5.5	6 hours- Implementing a screen in the angband game that shows the search function	3.5	18th
Date Update for edited monster	Sergio Emily	3 Hours	3.5	4- Application in the angband game correctly with the date updated	3 hours	February 20th
Creation of timer for start and stop, a clear variable that is accessible to screen updates	Sergio Connor chris	3	3.5	6- altering existing code	3.5	February 19th
Data Updating- for game character stats, both dealth and game character screens	Emily Elyse	3	3.5	6-Understanding the implemented timer code	3.5	February 22nd
Testing- Trouble shooting,	Chris	2.5	3	3- Communication in	3	February 23rd

attempting to break code, Providing feedback to Correct people for fixes or assistance with fixes				fixes could be a time drainer		
Instructors for running game document	Sergio	1	1.5	2- game not working	1.5	February 24th
[README.md](#) file with instructions for installing game,and example screenshots	Emily	1	1.5	2- game not working	1	February 24th
Account of time spent	Elyse	1	1.5	2- Poor Communication	1	February 24th
Man page of program- https://www.cyberciti.biz/faq/linux-unix-creating-a-manpage/	Connor	2	2.5	3- Poor instructions and help.	2	February 24th

Summary of time per person						
Person	Elyse	Emily	Sergio	Chris	Connor	Total together
Time per person (estimate)	11.5 hours	11 hours	11 hours	10 hours	11.5	55 hours individual (counts 2 people

						working together as 2 hours) Collaborative- 27 (counts 2 people working together as 1 hour)